Programming Language Translation – CSI248

Assignment II Report (Semester II 2025/2026)
Due Date: 06 April 2026
Group Members:

- Adam Musakabantu Muyobo – 202104060
- Zibisani Kgari Mholo – 2020055555
- Theo Kizito Tida – 202302071

1. Introduction

This report presents the implementation and analysis for Assignment II in CSI428 - Programming Language Translation. The main focus of the assignment was the construction of an LR(1) transition diagram, the derivation of an LR(1) parsing table, and the implementation of a modular Python parser that validates whether a given sentence belongs to the language defined by the supplied grammar. The report also answers the remaining theory questions in the same report style used in Assignment I.

For Question 1, the parser was implemented directly from the previously derived transition diagram and parsing table. We also attempted to use tokenization techniques learned in Assignment I, particularly the separation between lexical analysis and syntax analysis, the use of modular components, and clearer error categories. Although Assignment I focused on a different lexical problem, its emphasis on modularity and progressive processing influenced the structure of the final LR(1) parser.

2. Project Work Plan & Responsibilities

The project was completed collaboratively. The transition diagram and parsing table were derived manually and checked repeatedly through closure and goto computation before implementation.

- Adam: LR(1) item construction, parsing table derivation, parser architecture, documentation coordination.
- Zibisani: grammar validation, parser testing, debugging support, report review.
- Theo: transition checking, sample input testing, parser feedback, final review.
- All: discussion of grammar conflicts, final proofreading, and repository organization.

3. Question 1 - LR(1) Parsing, Parsing Table and Program

3.1 Grammar Ordering Used in the Parser

To make the LR(1) construction precise, the grammar was first ordered and numbered. The augmented grammar below is the exact ordering used when building the states, deriving reductions, and implementing the parser.

(0) S' -> S
(1) S -> X n1 C1 C2 G V N T L
(2) X -> x
(3) C1 -> c1
(4) C2 -> c2
(5) G -> g
(6) G -> ε
(7) V -> v
(8) N -> n T
(9) N -> ε
(10) T -> c3 c4 G V N
(11) T -> ε
(12) L -> l
(13) L -> ε

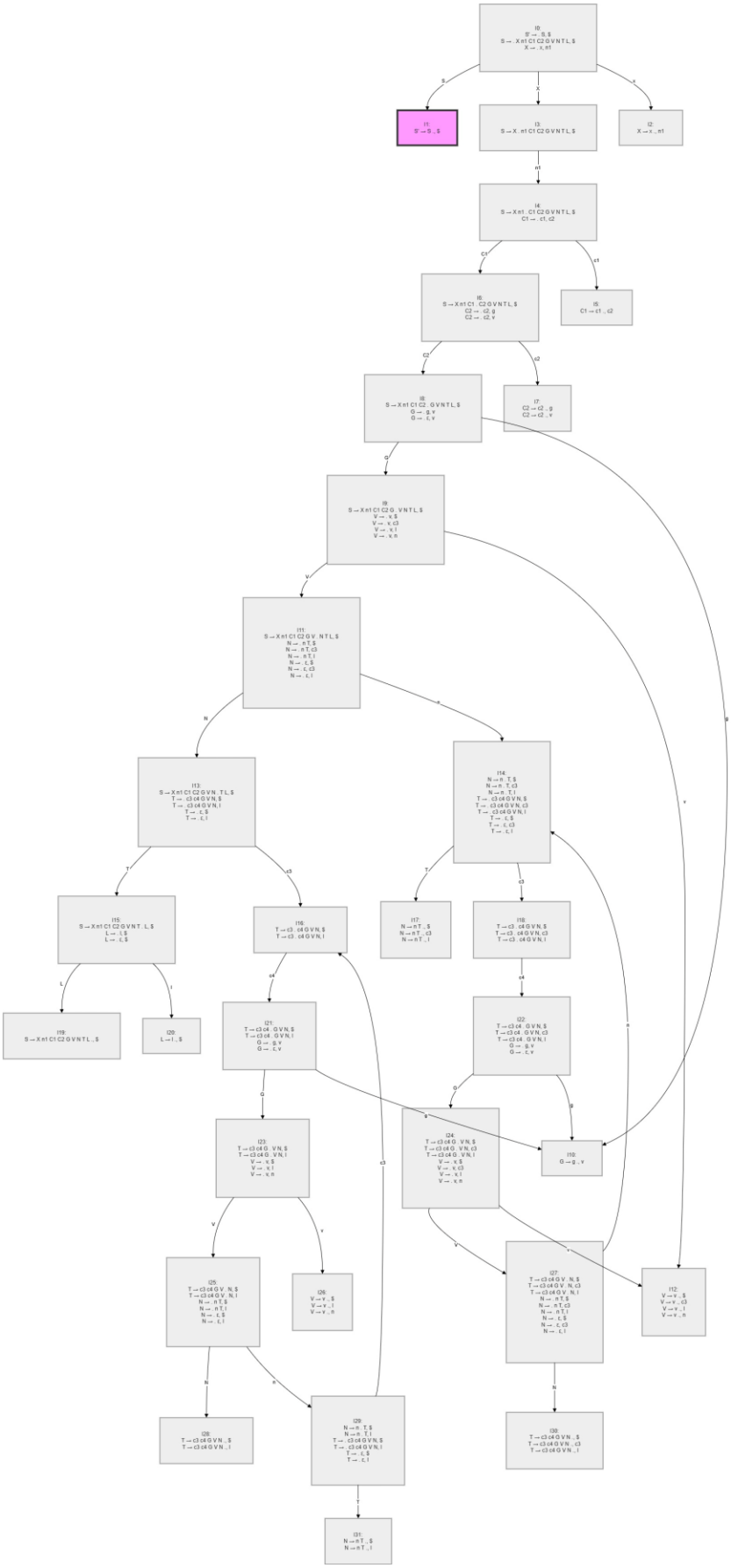
3.2 Explanation of the Ordering

The grammar was ordered so that each reduction in the parsing table could refer to a single production number. This ordering is important because LR parsing tables do not reduce by rewriting the entire production every time; instead, the parser performs actions such as r2, r8, or r11, each of which refers to one exact production in the ordered grammar. Using an augmented start production S' -> S also makes it possible to define the accept configuration formally.

The sequence of non-terminals in the main sentence rule, S -> X n1 C1 C2 G V N T L, reflects the expected sentence pattern in the language. The parser therefore expects tokens to appear in that same general order, while still allowing optional parts through ε-productions in G, N, T, and L.

3.3 LR(1) Transition Diagram

The canonical LR(1) collection of items was constructed using the closure and goto functions. Each state contains LR(1) items of the form A -> α . β, a, where a is the lookahead symbol. The final transition diagram was then derived from the goto transitions between these states.


I0:
S' → .S, $
S → .Xn1 C1 C2 G V N T L, $
X → .x, n1
S
X
x
I1:
S' → S., $
I3:
S → X. n1 C1 C2 G V N T L, $
I2:
X → x., n1
n1
I4:
S → Xn1. C1 C2 G V N T L, $
C1 → .c1, c2
C1
c1
I6:
S → Xn1 C1. C2 G V N T L, $
C2 → .C2, g
C2 → .c2, v
I5:
C1 → c1., c2
C2
c2
I8:
S → Xn1 C1 C2. G V N T L, $
G → .g, v
G → .c, v
I7:
C2 → c2., g
C2 → c2., v
g
I9:
S → Xn1 C1 C2 G. V N T L, $
V → .v, $
V → .v, c3
V → .v, l
V → .v, n
v
I11:
S → Xn1 C1 C2 G V. N T L, $
N → .n T, $
N → .n T, c3
N → .n T, l
N → .c, $
N → .c, c3
N → .c, l
N
n
I13:
S → Xn1 C1 C2 G V N. T L, $
T → .c3 c4 G V N, $
T → .c3 c4 G V N, l
T → .c, $
T → .c, l
T
c3
I14:
N → n. T, $
N → n. T, c3
N → n. T, l
T → .c3 c4 G V N, $
T → .c3 c4 G V N, c3
T → .c3 c4 G V N, l
T → .c, $
T → .c, c3
T → .c, l
T
c3
I15:
S → Xn1 C1 C2 G V N T. L, $
L → .L, $
L → .L, $
L
l
I16:
T → c3. c4 G V N, $
T → c3. c4 G V N, l
c4
I17:
N → nT., $
N → nT., c3
N → nT., l
I18:
T → c3. c4 G V N, $
T → c3. c4 G V N, c3
T → c3. c4 G V N, l
c4
I19:
S → Xn1 C1 C2 G V N T L., $
I20:
L → l., $
I21:
T → c3 c4. G V N, $
T → c3 c4. G V N, l
G → .g, v
G → .c, v
I22:
T → c3 c4. G V N, $
T → c3 c4. G V N, c3
T → c3 c4. G V N, l
G → .g, v
G → .c, v
G
g
I23:
T → c3 c4 G. V N, $
T → c3 c4 G. V N, l
V → .v, $
V → .v, l
V → .v, n
I24:
T → c3 c4 G. V N, $
T → c3 c4 G. V N, c3
T → c3 c4 G. V N, l
V → .v, $
V → .v, c3
V → .v, l
V → .v, n
I10:
G → g., v
c3
V
v
I25:
T → c3 c4 G V. N, $
T → c3 c4 G V. N, l
N → .n T, $
N → .n T, l
N → .c, $
N → .c, l
I26:
V → v., $
V → v., l
V → v., n
I27:
T → c3 c4 G V. N, $
T → c3 c4 G V. N, c3
T → c3 c4 G V. N, l
N → .n T, $
N → .n T, c3
N → .n T, l
N → .c, $
N → .c, c3
N → .c, l
I12:
V → v., $
V → v., c3
V → v., l
V → v., n
N
n
c
I28:
T → c3 c4 G V N., $
T → c3 c4 G V N., l
I29:
N → n. T, $
N → n. T, l
T → .c3 c4 G V N, $
T → .c3 c4 G V N, l
T → .c, $
T → .c, l
I30:
T → c3 c4 G V N., $
T → c3 c4 G V N., c3
T → c3 c4 G V N., l
T
I31:
N → nT., $
N → nT., l

3.4 Conflict Found in the LR(1) Table

While constructing the canonical LR(1) table, a shift/reduce conflict appeared in state 14 on lookahead c3. In that state, the parser can either shift on c3 to continue parsing T -> c3 c4 G V N, or reduce immediately using T -> ε. Because both actions are possible for the same state/lookahead pair, the grammar is not conflict-free canonical LR(1) under the table that was derived.

This conflict was not hidden or resolved manually inside the implementation. Instead, the Python program preserves the table exactly as derived and raises a grammar conflict error whenever execution reaches that conflicted entry. This keeps the implementation faithful to the manually constructed transition diagram and parsing table.

3.5 Parsing Table

The following ACTION and GOTO tables were implemented directly in the parser. Shift actions are written as sN, reductions as rN, and the accept state is marked as acc. The conflicted entry is intentionally shown as s18 / r11.

3.5.1 ACTION Table (States 0-15)

State	x	n1	c1	c2	g	v	n	c3	c4	l	$
0	s2										
1											acc
2		r2									
3		s4									
4			s5								
5				r3							
6				s7							
7					r4	r4					
8					s10	r6					
9						s12					
10						r5					
11							s14	r9		r9	r9
12							r7	r7		r7	r7
13								s16		r11	r11
14								s18 / r11		r11	r11
15										s20	r13

3.5.2 ACTION Table (States 16-31)

State	x	n1	c1	c2	g	v	n	c3	c4	l	$
16									s21		
17								r8		r8	r8
18									s22		
19											r1
20											r12
21					s10	r6					

22					s10	r6					
23						s26					
24						s12					
25							s29			r9	r9
26							r7			r7	r7
27							s14	r9		r9	r9
28										r10	r10
29								s16		r11	r11
30								r10		r10	r10
31										r8	r8

3.5.3 GOTO Table (States 0-15)

State	S	X	C1	C2	G	V	N	T	L
0	1	3							
1									
2									
3									
4			6						
5									
6				8					
7									
8					9				
9						11			
10									
11							13		
12									
13								15	

14								17	
15									19

3.5.4 GOTO Table (States 16-31)

State	S	X	C1	C2	G	V	N	T	L
16									
17									
18									
19									
20									
21					23				
22					24				
23						25			
24						27			
25							28		
26									
27							30		
28									
29								31	
30									
31									

3.6 Python Program Design

The final program is modular and follows the same separation-of-responsibilities style used in the previous assignment. The lexer converts words into grammar terminals such as x, n1, c1, c2, g, v, n, c3, c4, and l. The parsing engine then uses the hand-written ACTION and GOTO tables to perform shift, reduce, and accept operations.

The project is split into dedicated files for grammar definitions, tokenization, parsing tables, parser execution, and exception handling. This makes the system easier to test, explain, and maintain. The final implementation and source code are available in the GitHub repository: https://github.com/Adam-Muyobo/LR1_Parser

3.7 Results and Discussion

The parser can correctly report three distinct outcomes. First, it accepts valid input sentences when the token sequence follows the grammar and does not encounter a conflicted table entry. Second, it throws lexical errors when a word is not part of the supplied vocabulary. Third, it throws syntax errors when the token stream does not fit any valid parser action for the current state.

In addition, the parser can raise a grammar conflict error. This is an important result because it shows that the implementation remained faithful to the derived LR(1) table instead of forcing an arbitrary choice. Therefore, the program not only acts as a parser, but also as a validation tool for the correctness and determinism of the hand-constructed parsing table.

4. Question 2 - Natural Language Processing Concepts

4.1 Two Common Methods Used to Check Syntax in Natural Language Processing

i) Rule-Based (Grammar-Driven) Parsing

In this approach, a set of grammatical rules is defined based on the structure of a language. These rules describe how words and phrases should be arranged to form valid sentences. A parser then analyses an input sentence and checks whether it conforms to these predefined rules. For example, a rule in English may state that a sentence consists of a noun phrase followed by a verb phrase. If an input sentence does not follow this structure, a syntax error is identified.

ii) Statistical / Machine Learning-Based Parsing

In this approach, models are trained on large collections of text in order to learn the likelihood of different syntactic structures. For example, a probabilistic context-free grammar assigns probabilities to grammar rules based on how frequently they appear in training data. When analysing a new sentence, the parser selects the most probable parse tree. This method handles ambiguity more effectively than purely rule-based approaches because it relies on patterns observed in real language data.

4.2 Two Techniques to Determine the Intended Meaning of Words in Natural Language

i) Word Sense Disambiguation
Word Sense Disambiguation attempts to determine which meaning of a word is intended in a given context. This can be done through supervised methods, where a model is trained on labelled examples of different senses, or through knowledge-based methods, which rely on lexical resources such as WordNet. This is important when a word has multiple meanings and context is needed to determine the correct interpretation.

ii) Semantic Role Labelling
Semantic Role Labelling identifies the role that each word or phrase plays in the meaning of a sentence. It answers questions such as who performed an action, what action was performed, and what entity was affected. For example, in the sentence 'The student submitted the assignment', the student is the agent, submitted is the predicate, and the assignment is the theme. By identifying these roles, a system can better determine the intended meaning of an utterance.

5. Question 3 - Programming Language Theory and Java Concepts

5.1 How Compilers / Interpreters Handle Variables Without Declaration
In dynamically typed languages, the type of variable is determined at runtime rather than at compile time. When a variable is first assigned a value, the interpreter infers its type from the assigned value. For example, in Python, writing x = 5 creates a variable whose runtime value is an integer. The interpreter tracks the name, binding, and type in a symbol table as execution continues.

Many interpreters also use implicit declaration through symbol table management. When a variable name is encountered for the first time and assigned a value, an entry is created in the current scope. If the variable is referenced before assignment, most languages raise a runtime error. Python raises a NameError, while JavaScript historically allowed hoisting for variables declared with var, giving them an initial value of undefined.

5.2 Does the Following Code Compile in Java? Explain the Outcome
```
boolean b = false;
while (b) {
   // statements or empty.
}
```
Yes, this code compiles successfully in Java because the while statement requires a boolean condition and b is a valid boolean variable. There is no type error, so the compiler accepts the program.

At runtime, however, the loop body never executes because b is false from the beginning and its value is never changed. So the condition fails immediately and execution continues after the loop. The code is syntactically and semantically valid even though the loop body is effectively unreachable in practice.

5.3 Is 'A' Different from 'a' in Java?

Yes. Java identifiers are case-sensitive, so A and a are treated as two completely different identifiers. For example, int A = 1; and int a = 2; can exist in the same scope as separate variables.

5.4 Type of Result When Adding a Float and an Integer in Java

When a float and an integer are added together in Java, the result is of type float. This happens because Java performs implicit numeric promotion, widening the integer to float before carrying out the arithmetic operation.

5.5 Narrowing Types in Java

Narrowing type conversion in Java is the process of converting a value from a wider type to a narrower type, such as from double to int. Because information may be lost during this conversion, narrowing usually requires an explicit cast.

6. Question 4 - Characteristics of Languages and Domain Suitability

6.1 Characteristics That Make a Quality Programming Language

- Readability: Programs should be easy to read and understand.
- Writability: The language should make it easy to express correct solutions.
- Reliability: Programs should behave consistently and predictably.
- Expressiveness: Complex ideas should be representable clearly and compactly.
- Portability: Programs should be usable across different systems with minimal change.
- Efficiency: The language should support effective use of time and memory resources.
- Orthogonality: Language features should combine in consistent ways.
- Abstraction Support: The language should support functions, classes, modules, and other abstraction mechanisms.
- Exception Handling: Runtime problems should be handled in a structured way.
- Simplicity: The language should avoid unnecessary complexity and be teachable.

6.2 Languages for Scripting vs Data Processing

Scripting Languages

Scripting languages are usually interpreted and are therefore quick to write, test, and modify. They are commonly used for automation, file processing, operating system interaction, and rapid development.

Bash/Shell Script is widely used on Unix/Linux systems for automating administrative tasks such as file handling, backups, and process control. JavaScript, especially through Node.js, is widely used for web automation, APIs, and event-driven server-side scripting.

Data Processing Languages
Data processing languages are designed to work efficiently with large volumes of structured or numerical data. They usually provide strong library support for computation, analysis, and visualization.

Python is widely used because of libraries such as NumPy, pandas, and matplotlib. R is especially strong for statistics and research-oriented analysis. SQL is specialized for storing, querying, and manipulating relational data efficiently.

What makes a language suited for a particular domain?
A language becomes suitable for a domain because of its design goals and ecosystem. For scripting, ease of writing, quick execution, and close system integration are important. For data processing, expressive syntax, specialized libraries, and strong support for large datasets are essential. Domain-specific languages such as SQL are highly effective within a narrow problem space, while general-purpose languages such as Python succeed across multiple domains because of rich library ecosystems.

7. Conclusion

This assignment combined formal parsing techniques with implementation and theory. The LR(1) transition diagram and parsing table provided a concrete example of how syntax analysis can be built from grammar rules, while the Python program demonstrated how those results can be translated into an executable parser. A key finding was the shift/reduce conflict in state 14, which showed that the derived table is not conflict-free canonical LR(1). Preserving that conflict in the implementation made the parser more academically faithful and made the report more honest about the grammar's behavior.

The remaining theory questions further reinforced ideas from natural language processing, variable handling, Java typing, and language design. Overall, the assignment connected formal theory, parser construction, and practical implementation in a consistent workflow.

GitHub Repository

LR(1) Parser repository: https://github.com/Adam-Muyobo/LR1_Parser